Техническая документация по реверс-инжинирингу прошивок ППКП-01Ф

Опорный документ для проектирования новой прошивки и продолжения исследования кода

Основание: анализ файлов A03_26.PZU, A04_28.PZU, ppkp2012 a01.PZU, ppkp2019 a02.PZU;

инструкция «Конфигуратор программ ППКП-01Ф-20.00…15»;

проектная документация из архива «Главный корпус.zip».

Дата подготовки: 22.04.2026

Статус документа: рабочая техническая база, допускающая уточнение по мере продолжения реверса.

Содержание

1. Назначение документа

2. Использованные материалы и ограничения

3. Идентификация исследованных файлов и привязка к проекту

4. Общая картина исследуемой платформы

5. Формат файлов .PZU и общие технические признаки

6. Архитектура прошивки

7. Карта памяти и интерпретация ключевых областей XDATA/ROM

8. Объектная модель прошивки

9. Подсистема интерфейса и текстовых строк

10. Подсистема событий, очередей и пакетного обмена

11. Сопоставление с проектной документацией объекта

12. Выводы для проектирования новой прошивки

13. Практические рекомендации по продолжению реверса

14. Каталог ключевых функций и гипотез

15. Приложение А. Таблица уверенности выводов

1. Назначение документа

Документ фиксирует текущее состояние реверс-инжиниринга прошивок приборов ППКП-01Ф, полученное по четырём образам ПЗУ, инструкции к конфигуратору и проектной документации. Его назначение двоякое: во-первых, дать единую техническую базу для проектирования новой прошивки; во-вторых, зафиксировать уже доказанные выводы и рабочие гипотезы, чтобы дальнейшее исследование кода можно было вести последовательно, не теряя контекст.

Документ не является официальной документацией ПАО «Тензор». Все выводы получены методом анализа доступных файлов и сопоставления с проектными схемами. Для удобства практического применения выводы разделены на три класса: подтверждённые, вероятные и требующие дальнейшей проверки.

2. Использованные материалы и ограничения

2.1. Исходные материалы

Материал	Назначение в анализе
A03_26.PZU	образ ПЗУ/прошивки, связанный с ППКП-01Ф-26
A04_28.PZU	образ ПЗУ/прошивки, связанный с ППКП-01Ф-28 (A.04)
ppkp2012 a01.PZU	другая ветка прошивки, вероятно более общая/платформенная
ppkp2019 a02.PZU	более поздняя ветка прошивки, содержит строку Ver.2013s/ RTOS
Конфигуратор ПЗУ.doc	инструкция по работе с конфигуратором программ ППКП-01Ф-20.00…15
Главный корпус.zip	комплект проектной документации, позволивший

	привязать часть прошивок к конкретным приборам проекта

2.2. Ключевые ограничения

– Нижняя область ПЗУ ниже адреса 0x4000 в анализируемых .PZU отсутствует. По вызовам на адреса ниже 0x4000 видно, что существует общий системный ROM-слой, которого в распоряжении нет.

– Нет штатного описания транспортного протокола RS-485/RS-232 и нет полной карты регистров/объектов.

– Часть выводов по семантике полей записи объекта и по кодам событий остаётся гипотетической, хотя архитектура в целом уже понятна.

– Документ конфигуратора относится к вариантам 20.00…15, а не прямо к 20.21. Однако его модель 8-байтных буферов хорошо согласуется с внутренними структурами прошивки.

3. Идентификация исследованных файлов и привязка к проекту

По совокупности строковых меток в прошивках и обозначений на проектных листах установлено, что файлы A03_26.PZU и A04_28.PZU не являются абстрактными версиями ПО. Они привязаны к конкретным приборам проекта.

Файл	Встроенная метка	Привязка к проекту	Вывод
A03_26.PZU	A.03 Serovskaq	ППКП-01Ф-26, лист 14/4	Объектно-привязанная прикладная прошивка
A04_28.PZU	A.04 Serovskaq	ППКП-01Ф-28 (A.04), листы 14/5–14/6	Объектно-привязанная прикладная прошивка
ppkp2012 a01.PZU	ppkp2012 A.01	прямая привязка к текущему проекту не подтверждена	вероятно системная/платформенная ветка
ppkp2019 a02.PZU	ppkp2019 A.02, Ver.2013s/ RTOS	прямая привязка к текущему проекту не подтверждена	более поздняя платформенная ветка

4. Общая картина исследуемой платформы

Анализ показал, что прошивки принадлежат одной технологической семье, но минимум двум веткам развития. Ветка A03/A04 представляет объектно-привязанное прикладное ПО под конкретные приборы и схему объекта. Ветка ppkp2012/ppkp2019 выглядит как более общая платформа с развитым интерфейсом и системным слоем.

По набору опкодов и по характерным конструкциям код с высокой вероятностью относится к архитектуре 8051/MCS-51 или совместимой с ней. Это подтверждается наличием инструкций LJMP, LCALL, RETI, MOV DPTR,#data16, MOVX A,@DPTR, MOVX @DPTR,A, DIV AB, DA A и типичных схем работы с DPTR.

4.1. Что уже понятно по модели работы

• прошивка событийно-ориентированная, а не просто циклически «льёт» все данные наружу;

• существуют большие блоки состояния, которые зеркалируются и сравниваются на изменение;

• существует короткий событийный/командный канал отдельный от больших блоков состояния;

• есть объектная таблица фиксированного размера с индексируемыми записями;

• есть битовые карты активных/разрешённых объектов;

• есть текстовый интерфейс с таблицей строк и форматированием чисел;

• есть пакетный транспортный слой верхнего уровня, а финальная отправка делегируется в нижний ROM-сервис.

5. Формат файлов .PZU и общие технические признаки

5.1. Формат контейнера

Все четыре файла .PZU представляют собой текстовые Intel HEX-образы.

Параметр	A03_26	A04_28	ppkp2012/2019
Формат	Intel HEX	Intel HEX	Intel HEX
Объём данных	32768 байт	32768 байт	32768 байт
Диапазон адресов	0x4000–0xBFFF	0x4000–0xBFFF	0x4000–0xBFFF
Завершающая запись	:00000001FF	:00000001FF	:00000001FF

5.2. Контрольная сумма

У всех исследованных образов последние два байта соответствуют 16-битной аддитивной контрольной сумме всех предыдущих байтов образа по модулю 65536. Непосредственно перед контрольной суммой в конце файла стоит байт 0x01. Это позволяет сделать практический вывод: при любом ручном изменении образа контрольную сумму необходимо пересчитывать, иначе загрузчик или сама программа могут отвергнуть изменённый ПЗУ-образ.

6. Архитектура прошивки

6.1. Общая схема

– 1) нижний ROM-слой ведёт часть общих сервисов, драйверов и, вероятно, транспорт/монитор;

– 2) прикладной слой в диапазоне 0x4000–0xBFFF работает с объектами, состояниями, интерфейсом и очередями обмена;

– 3) большие снимки состояния сравниваются с зеркалами;

– 4) по изменению ставятся биты очереди;

– 5) по каждому типу изменения формируется свой пакет;

– 6) пакет передаётся в нижний сервис через дескриптор/буфер в области 0x5006+;

– 7) текстовый интерфейс и сценарии пользователя обслуживаются параллельно.

6.2. Стартовый участок

В начале образов присутствует таблица длинных переходов. Для A04_28.PZU она выглядит как переходы с 0x4000 на 0x4100, 0x4176, 0x41D0, 0x492E, 0x4954, 0x497A. Это похоже на стартовые обработчики и векторы.

```
4000: LJMP 4100h
4006: LJMP 4176h
400C: LJMP 41D0h
4012: LJMP 492Eh
4018: LJMP 4954h
401E: LJMP 497Ah
```

7. Карта памяти и интерпретация ключевых областей XDATA/ROM

Ниже приведена рабочая карта адресов, достаточная для проектирования новой прошивки и продолжения реверса. Для каждого адреса указан смысл, который либо подтверждён кодом, либо является лучшей текущей гипотезой.

Адрес/область	Назначение	Статус вывода	Комментарий
0x30C0	текстовый/рабочий буфер	подтверждено	используется подсистемой вывода строк и как scratch-зеркало
0x3297	индекс/длина текстового буфера	подтверждено	инкрементируется при добавлении символов
0x329C	очередь изменений	подтверждено	битовая маска событий/пакетов
0x329D	активный селектор передачи	подтверждено	state machine арбитра обмена
0x3295	индекс сканирования объектов	вероятно	циклический обход объектов
0x3298	текущий выбранный объект	подтверждено	индекс текущей записи объекта
0x32A2	код служебного события/режима	подтверждено	значения 02, 83, 84, 85 и др.
0x329F	текущая копия кода события	подтверждено	сравнивается с 0x32A2
0x3153..3155	новый 3-байтовый payload события	подтверждено	сравнивается с 0x3150..3152
0x3150..3152	рабочая копия 3-байтового payload	подтверждено	при изменении ставится bit3 очереди
0x3110	большой блок состояния №1	подтверждено	зеркалируется и передаётся отдельным пакетом
0x3128	большой блок состояния №2	подтверждено	зеркалируется и передаётся отдельным пакетом
0x3317..3336	битовая карта активных объектов	подтверждено	32 байта = 256 бит
0x328E	битовая карта выбора/состояния	вероятно	используется в UI-сценариях
0x315A	связанная битовая карта	вероятно	очищается/ устанавливается вместе с 0x328E
0x4DB2	битовая карта допустимых объектов	вероятно	используется при выборе валидного объекта
0x343D + index*16	таблица записей объектов	подтверждено	размер записи 16 байт
0x5006+	дескриптор/буфер обмена нижнего слоя	вероятно	передаётся в системную функцию 0x0127
0x5008+	заголовок и полезная нагрузка пакета	подтверждено	собирается builder-функциями
0x6176	таблица указателей на строки	подтверждено	используется для вывода текста

8. Объектная модель прошивки

Ключевой результат реверса состоит в том, что прошивка устроена не вокруг разрозненных дискретных сигналов, а вокруг массива логических объектов. Это объясняет, как один и тот же движок мог применяться на приборах с разным прикладным наполнением.

8.1. Размер и адресация записи объекта

Запись объекта имеет размер 16 байт. Текущий выбранный объект адресуется через базу 0x343D и индекс 0x3298, умноженный на 16. Отдельно существует индекс 0x3295, используемый как сканер/поисковик следующего подходящего объекта.

8.2. Поля записи объекта

Смещение	Рабочая интерпретация	Статус	Основание
+0	тип/класс объекта	вероятно близко к подтверждённому	ветвления по значению 1 и другим значениям
+1	компактный код состояния	вероятно	используется как часть упакованного статуса
+2	индекс связанного ресурса/группы	вероятно	уменьшается на 1 и служит индексом таблиц
+3	режим объекта, младшие 2 бита значимы	вероятно	в UI пишутся 0/1, в других местах маскируется ANL #03
+4	параметр/уровень/ настройка объекта	гипотеза	перебирается через таблицу допустимых значений
+5	ссылка на связанный объект/группу	гипотеза	используется при переходе к другому объекту
+6	дополнительный статусный байт	гипотеза	формируется из битовых данных и записывается обратно
+7..+15	не расшифровано	неизвестно	требуется дальнейшее исследование по использованию

8.3. Роль индексов 0x3295 и 0x3298

0x3298 следует считать индексом текущего выбранного объекта. 0x3295 — это индекс сканирования, который циклически обходит объекты и помогает выбрать следующий допустимый объект. Функция 0x7881 нормализует текущий выбор, используя карты допустимости и дополнительные критерии.

9. Подсистема интерфейса и текстовых строк

9.1. Буфер текста и вывод строк

Текстовая подсистема имеет отдельный буфер 0x30C0 и индекс 0x3297. Функция 0x6FB8 добавляет один символ, 0x6F83 очищает буфер, а 0x7020 печатает строку из кодовой памяти до байта 0xFF. Таблица указателей на строки располагается около 0x6176.

```
6FB8h  - append_char(buffer=0x30C0, idx=0x3297)
6F83h  - clear_text_buffer()
7020h  - print_rom_string_until_FF(DPTR)
6E89h  - select_string_by_index_via_pointer_table(0x6176)
```

9.2. Форматирование чисел

В коде присутствуют отдельные функции форматирования шестнадцатеричных и десятичных цифр, а также преобразование 16-битных значений в BCD. Это показывает, что интерфейс прибора и диагностика поддерживают вывод не только строковых сообщений, но и числовых параметров.

10. Подсистема событий, очередей и пакетного обмена

10.1. Детектор изменений

Функция 0x8808 сравнивает новые и старые копии крупных блоков и коротких служебных полей. Если обнаружено изменение, соответствующий фрагмент копируется в рабочую копию, а в 0x329C устанавливается бит очереди.

```
if block_3110_changed: queue.bit0 = 1
if block_3128_changed: queue.bit1 = 1
if code_32A2_changed: queue.bit2 = 1
if payload_3153_changed: queue.bit3 = 1
```

10.2. Классы пакетов

Для каждого класса изменений существует свой builder-функция. Это позволяет считать, что обмен организован пакетами нескольких типов, а не единым неструктурированным буфером.

Функция	Что передаёт	Размер/характер	Статус
0x8999	блок 0x3110 -> 0x500F+	крупный пакет	подтверждено
0x89B1	блок 0x3128 -> 0x500F+	крупный пакет	подтверждено
0x89C9	байт 0x329F -> 0x500F	малый пакет	подтверждено
0x89DD	0x3150..3152 -> 0x5010+	малый 3-байтовый пакет	подтверждено
0x8989	служебный/стартовый пакет	очень малый	вероятно

10.3. Арбитр обмена

Функции 0x889F и 0x890E образуют конечный автомат транспортного уровня верхнего ROM-слоя. Они просматривают очередь, выбирают следующий тип пакета, вызывают builder и затем передают дескриптор/буфер в нижний системный вызов 0x0127. При этом используются флаг занятости и короткий интервал между передачами.

10.4. Короткий служебный канал

Связка 0x32A2 + 0x3153..3155 образует короткий событийный/командный канал. 0x32A2 содержит код события, а 0x3153..3155 — payload. Событие сравнивается, пакуется и отправляется отдельным типом пакета. Найдены значения кодов 0x02, 0x83, 0x84, 0x85.

Особенно интересен сценарий, при котором устанавливается код 0x85 и payload 55 55 55, после чего через таймер состояние сбрасывается. Это очень похоже на импульсную служебную команду или кратковременный управляющий маркер.

11. Сопоставление с проектной документацией объекта

11.1. ППКП-01Ф-26

По проекту прибор ППКП-01Ф-26 связан с основным и резервным RS-485, вызовами к шкафам и управлением огнезадерживающими клапанами. Это согласуется с тем, что A03_26.PZU выглядит как сравнительно компактная объектно-привязанная прикладная прошивка.

11.2. ППКП-01Ф-28

По проекту ППКП-01Ф-28 (A.04) обслуживает рабочий и резервный пожарные насосы, несколько направлений, сигналы «Открыть», «Закрыть», «Открыто», «Закрыто», «Вода пошла», «Нет питания», «Обрыв фаз», «Заклинено», а также статусы «Авт. режим», «Неисправность», «Насос включен». Это хорошо согласуется с объектной моделью прошивки, битовыми картами активности и крупными блоками состояния 0x3110 / 0x3128.

11.3. Практический вывод для новой прошивки

При проектировании новой прошивки не следует жёстко кодировать каждый дискретный сигнал отдельной логической веткой. Исследованная прошивка явно использует объектную модель, где насосы, направления, аварии и режимы представляются логическими объектами с типом, состоянием, режимом и связями. Повторение этой архитектуры повысит совместимость с будущим реверсом и упростит масштабирование конфигурации.

12. Выводы для проектирования новой прошивки

- Необходимо сохранить разделение на прикладной слой и транспортный/системный слой.
- Рекомендуется использовать таблицу объектов фиксированного формата, а не разрозненные переменные.
- Следует отделить крупные телеметрические блоки от коротких событийных/командных сообщений.
- Нужно предусмотреть битовые карты активности/разрешения для быстрого поиска активных объектов.
- Состояния должны формироваться как зеркала с детектором изменений, чтобы передавать только изменившиеся данные.
- Текстовый интерфейс и диагностика должны использовать отдельную таблицу строк и форматтер чисел.
- Конфигурационные данные должны быть отделены от кода и доступны через понятный сервисный интерфейс.

12.1. Предлагаемая модульная структура новой прошивки

Модуль	Содержание
HAL/Drivers	аппаратные абстракции портов, линий, таймеров, UART/RS-485, EEPROM/flash
Transport Core	очереди, пакеты, CRC/контрольные суммы, таймеры обмена, подтверждения
Object Model	массив объектов фиксированного размера, типы, состояния, режимы, связи
Process Logic	насосы, направления, клапаны, резерв, аварии, блокировки
UI/Diagnostics	экран, строки, журнал, сервисные сообщения, считывание параметров
Configurator API	чтение/запись конфигурации, экспорт/импорт, сервисные команды

13. Практические рекомендации по продолжению реверса

– Прежде всего нужно добить поля записи объекта по смещениям +4…+15: искать все функции, которые читают и пишут эти поля.

– Нужно построить перечень всех обращений к базе 0x343D + index*16 и сгруппировать их по смещениям поля.

– Следует отдельно расшифровать таблицу 0x7430 и область 0x3165, так как они выглядят как связи «группа -> подэлементы».

– Нужно продолжать анализ кодов события 0x32A2, начиная с 0x02, 0x83, 0x84, 0x85 и всех мест, где эти значения ставятся и сбрасываются.

– Для восстановления проводного протокола требуется найти или считать нижний ROM-слой ниже 0x4000 либо получить трафик с реального прибора.

– Если будет доступен прибор, необходимо сделать логирование RS-232/RS-485 при известных сценариях: выбор объекта, активация, сброс, смена режима, аварийное событие.

– Рекомендуется сделать отдельный каталог сигнатур функций с рабочими именами и использовать единое именование при дальнейшем реверсе.

14. Каталог ключевых функций и гипотез

Адрес	Рабочее имя	Назначение	Статус
0x6717	read_8byte_record	чтение 8-байтной записи из XDATA в R0..R7	подтверждено
0x6826	write_8byte_record	запись 8-байтной записи из R0..R7 в XDATA	подтверждено
0x69FA	xmem_copy	копирование блока между областями XDATA	подтверждено
0x6A12	xmem_compare	сравнение двух блоков XDATA	подтверждено
0x68DE	bit_index_to_addr_mask	получение адреса байта и битовой маски по индексу бита	подтверждено
0x687B	bit_test	проверка бита в битовой карте	подтверждено
0x6884	bit_clear	сброс бита в битовой карте	подтверждено
0x68A6	bit_set	установка бита в битовой карте	подтверждено
0x68B0	bit_toggle	инвертирование бита	подтверждено
0x6F83	clear_text_buffer	очистка текстового/рабочего буфера	подтверждено
0x6FB8	append_char	добавление символа в текстовый буфер	подтверждено
0x7020	print_rom_string	вывод строки из ROM до 0xFF	подтверждено
0x6E89	print_string_by_index	выбор строки по индексу через таблицу 0x6176	подтверждено
0x8808	change_detector	детектор изменений крупных блоков и короткого канала	подтверждено
0x889F	tx_scheduler	арбитр отправки/межпакетный таймер	вероятно
0x890E	tx_dispatcher	диспетчер очереди и выбор builder-функции	подтверждено
0x8989	build_default_packet	сборка служебного/начального пакета	вероятно
0x8999	build_packet_group1	упаковка блока 0x3110	подтверждено
0x89B1	build_packet_group2	упаковка блока 0x3128	подтверждено
0x89C9	build_packet_event_code	упаковка 1-байтового кода события	подтверждено
0x89DD	build_packet_event_payload	упаковка 3-байтового payload	подтверждено
0x89F9	build_packet_header	сборка заголовка пакета в 0x5008+	подтверждено
0x6BD0	copy_temp_payload_to_3153	запись 3 байт из 00..02 в 0x3153..3155	подтверждено

0x7881	normalize_current_object	поиск ближайшего допустимого объекта	вероятно
0x7991	ui_activate_object	сценарий активации объекта, field+3 = 1	вероятно
0x7AF8	ui_deactivate_object	сценарий снятия/сброса объекта, field+3 = 0	вероятно
0x804C	read_object_status_nibble	чтение status code объекта и сборка упакованного статуса	гипотеза близкая к подтверждённой

15. Приложение A. Таблица уверенности выводов

Уровень	Как использовать	Примеры
Подтверждено	можно опираться при проектировании и именовании функций/структур	формат Intel HEX, диапазон 0x4000–0xBFFF, 16-байтная таблица объектов, 8-байтные записи, таблица строк, очередь изменений
Вероятно	можно использовать как рабочую модель, но оставлять в коде комментарий FIXME/VERIFY	назначение 0x3295, 0x4DB2, поле +3 объекта как режим, tx_scheduler вокруг 0x889F
Гипотеза	использовать только как ориентир для дальнейшего реверса, не зашивать как окончательный факт	семантика field+4..+6, точный смысл кодов 0x83/0x84/0x85, распределение байтов 0x3110/0x3128 по конкретным сигналам проекта

Заключение

На текущем этапе получено достаточно материала, чтобы использовать этот документ как техническую опору для двух задач: разработки новой прошивки с похожей архитектурой и продолжения реверс-инжиниринга существующих образов. Ключевой вывод состоит в том, что прошивка организована вокруг объектной модели, битовых карт активности, больших блоков состояния, короткого событийного канала и пакетного обмена через нижний системный ROM-слой. При дальнейшем исследовании приоритет следует отдавать расшифровке полей объектной записи и привязке индексов объектов к конкретным насосам, направлениям, клапанам и аварийным цепям проекта.